

# Simple Feedforward - Backpropagation Neural Network

This script implements a simple feedforward artificial neural network (ANN) trained using the backpropagation algorithm. It approximates the function:

$$f(x) = (\sin(x) + 1) / 2$$

The ANN consists of a single hidden layer and is trained to minimize the mean squared error (MSE) between predicted and actual values.

This script serves as a reference implementation to help users understand how a simple feedforward backpropagation ANN can be implemented and modify it for more advanced applications. It can be extended to handle more than one input variables.

No MATLAB toolbox is required to run this code, which is particularly useful for educational NN prototypes or if you want fine-grained control over weight updates, learning rate, activation functions, etc. No dependencies on MATLAB's Deep Learning Toolbox or any other toolboxes exist, therefore it can run on any MATLAB version. It is transparent and easy to extend (ideal for NN research and learning). The local functions used in this script are listed alphabetically at the end of the main script.

See the file readMe.pdf included in this package for a detailed description.

Note: The results produced by the code may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. For consistency purposes and without loss of generality, the random number generator is appropriately initialized at the beginning of the code. If this option is removed, consider running the example a few times and compare the average outcome.

This code is licensed under [CC BY 4.0](#). This license allows reusers to distribute, remix, adapt, and build upon the material in any medium or format, even for commercial purposes. It requires that reusers give credit to the creator.

## Contents

---

- [Set up the training data](#)
- [Specify the prediction point](#)
- [Settings](#)
- [Process](#)
- [Output](#)

## Set up the training data

---

Define the function range and generate input-output pairs for training.

```
rng(0) % for reproducibility
k = 5; % Number of sine wave periods
x = linspace(0, k * 2 * pi, 500)'; % Input values
y = (sin(x(:,1)) + 1) / 2; % Target output values
```

## Specify the prediction point

---

```
xq=2; % Example input to test ANN after training
yq=(sin(xq)+1)/2; % Expected target output
```

## Settings

---

Hidden layer size

```
hidden_neurons=2^4;
```

Maximum number of epochs

```
max_Epochs = 20000;
```

Learning rate

```
alr = 0.01;  
blr = alr / 10;
```

Training error tolerance

```
train_err=0.001;
```

## Process

Check if x and y contain same number of observations

```
nObs=size(x,1);  
if nObs ~= size(y,1)  
    error('Data mismatch')  
end
```

Standardize the data to mean=0 and standard deviation=1

```
mu_x = mean(x);  
sigma_x = std(x);  
mu_y = mean(y);  
sigma_y = std(y);  
x_norm = (x - repmat(mu_x,nObs,1)) ./ repmat(sigma_x,nObs,1);  
y_norm = (y - repmat(mu_y,nObs,1)) ./ repmat(sigma_y,nObs,1);
```

Shuffle data

```
rand_idx = randperm(nObs);  
x_norm1 = x_norm(rand_idx,:);  
y_norm1 = y_norm(rand_idx,:);
```

Split data into training (80%) and validation (20%)

```
num_train = round(0.8 * nObs);  
train_inp = x_norm1(1:num_train,:);  
train_out = y_norm1(1:num_train,:);  
val_inp = x_norm1(num_train+1:end,:);  
val_out = y_norm1(num_train+1:end,:);
```

Add bias as an input

```
train_bias = ones(num_train,1);
val_bias = ones((nObs-num_train),1);
train_inp = [train_inp train_bias];
val_inp = [val_inp val_bias];
```

Number of input variables (including bias)

```
inputs = size(train_inp,2);
```

Set random initial weights

```
weight_input_hidden = randn(inputs,hidden_neurons)/10;
weight_hidden_output = randn(1,hidden_neurons)/10;
```

Main training loop

```
% Loop over epochs
for iter = 1:max_Epochs

    % Loop over patterns
    for j = 1:num_train

        % Set the current pattern
        this_pat = train_inp(j,:);
        act = train_out(j,1);

        % Calculate the error for this pattern
        hval = (tanh(this_pat*weight_input_hidden))';
        pred = hval'*weight_hidden_output';
        error= pred - act;

        % Adjust weight hidden - output
        delta_HO = error.*blr .*hval;
        weight_hidden_output = weight_hidden_output - delta_HO';

        % Adjust the weights input - hidden
        delta_IH= alr.*error.*weight_hidden_output'.*(1-(hval.^2))*this_pat;
        weight_input_hidden = weight_input_hidden - delta_IH';

    end

    % Training error
    pred_train = weight_hidden_output*tanh(train_inp*weight_input_hidden)';
    error_train = pred_train' - train_out;
    err_train(iter) = (mean(error_train.^2))^0.5;

    % Validation error
    pred_val = weight_hidden_output*tanh(val_inp*weight_input_hidden)';
    error_val = pred_val' - val_out;
    err_val(iter) = (mean(error_val.^2))^0.5;

    if mod(iter, 500) == 0
        fprintf('Epoch: %d RMSE (Train): %f, RMSE (Val): %f\n', iter,...
```

```

        err_train(iter), err_val(iter));
    end

    % stop if error is small
    if err_train(iter) < train_err
        fprintf('Converged at epoch: %d\n',iter);
        break
    end
end
end

```

Actual and predicted output at query point

```

xq_norm = (xq - repmat(mu_x,size(xq,1),1)) ./ repmat(sigma_x,size(xq,1),1);
pred_x = weight_hidden_output*tanh([xq_norm,ones(size(xq_norm,1),1)]*...
    weight_input_hidden)';
pred_yq = pred_x'* sigma_y + mu_y;

```

## Output

Display actual and predicted output of data input

```

a = y;
pred_x = weight_hidden_output*tanh([x_norm,ones(size(x_norm,1),1)]*...
    weight_input_hidden)';
b = pred_x'* sigma_y(:,1) + mu_y(:,1);
figure(1)
plot(x(:,1),b)
hold on
plot(x(:,1),a)
hold off
legend('Predicted','Actual')
xlabel('x_1')

```

Display training and validation error

```

figure(2)
semilogy(err_train)
hold on
semilogy(err_val)
hold off
legend('Training error','Validation error')
xlabel('Epoch')

```

Display actual and predicted output at query point

```

fprintf('Actual value at query point: %d\n', yq);
fprintf('Predicted value at query point: %d\n', pred_yq);

```